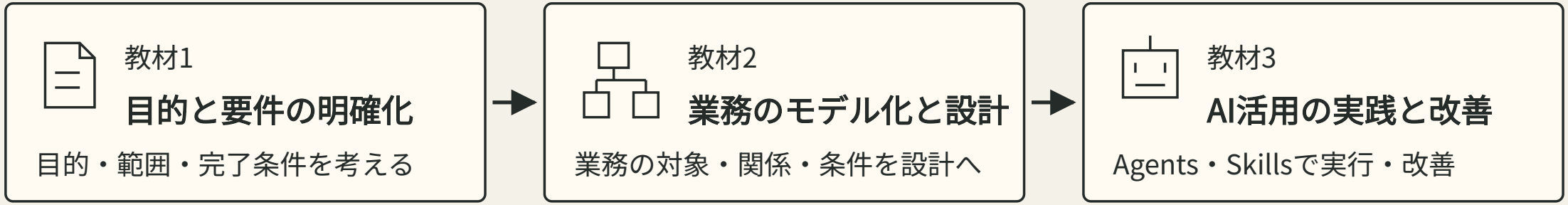
はじめに

経費精算システムの刷新を題材に、三つの教材の学びを紹介します。



AI活用で、開発全体の生産性と品質を改善する

経費精算システムの刷新を題材に、必要な品質と、案件全体の工数・期間を考えます。

経費精算システム刷新チーム

既存メンバー



PM・PL　開発担当　業務担当

新しく参加



AI

開発者

要件・設計からチーム全体でAIを活用

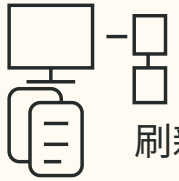
今回の刷新



引き継ぐ業務ルール

旧仕様・既存コードを確認

見直す構成と更新する資料



画面・API・業務処理

刷新後の仕様・設計も更新

チームの目標：確認・修正も含めた開発全体の改善



必要な品質を満たす



工数を減らし、開発期間を短くする

AIで、実装からレビュー承認まで短時間で進みました

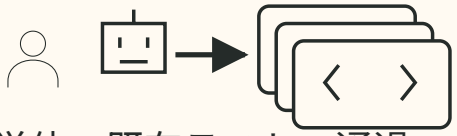
仕様・設計・コード・テストにAIを使い、変更を取り込む前のレビュー承認まで進みました。

旧仕様・既存コード
開発者が確認

作業条件：承認後は変更不可

短時間で進んだ範囲

設計・実装・テスト



単体・既存テスト：通過

AIと人のレビュー

AI：指摘なし

人：条件・差分・テスト

レビュー承認

PR：承認後は変更不可
承認段階は記載なし

取り込み前に承認

取り込み後：結合テスト

申請から上長承認・経理確認まで、業務の流れを確認

業務上必要な条件が抜け、不具合と再作業が発生しました

上長承認後に申請内容を変更できる不具合が結合テストで発覚し、修正と再確認が必要になりました。

本来の仕様

上長承認後から変更不可

申請（変更可） → 上長承認（変更不可） → 経理確認（変更不可）

12,000円 → 18,000円に変更

12,000円のまま　保存不可

今回の実装

結合テストで発覚

経理確認後から変更不可

申請（変更可） → 上長承認（変更可） → 経理確認（変更不可）

12,000円 → 18,000円に変更

18,000円に変更　保存された

取り込み前のテストで未確認：上長承認後・経理確認前の変更

変更制限を修正 → テストケース追加 → 再テスト・再レビュー

動作を直しても、設計の問題は残りました

変更制限を直した後も、上長承認と経理確認を区別しない設計のまま、追加対応を進めました。

修正済みの動作

上長承認後は直接変更できない

再テスト・再レビューで確認

残った設計

上長
支出承認

経理
証憑確認

承認済み：一つの状態
二つの判断の違いを表せない

設計を見直さず条件を追加

差し戻し
上長承認を維持
テスト

変更制限
差し戻し後は証憑のみ編集
テスト

再提出
経理確認を再開
テスト

確認の負担

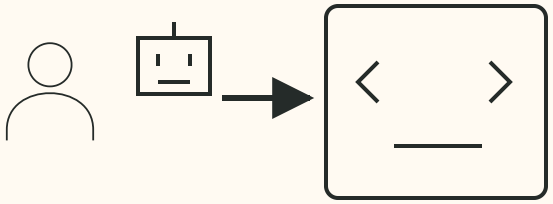
処理とテストを
読み合わせる
全体の関係を照合

証憑だけ修正する場合：上長承認は維持し、経理確認をやり直す

設計を見直さず変更を重ねると、確認範囲が広がる可能性があります

ここからは想定例です。設計を見直さず機能追加を続けた場合に、調査や確認の範囲がどう変わるかを考えます。

別の担当者が実装



関連する処理を把握しきれず
AIで似た処理を新しく作る

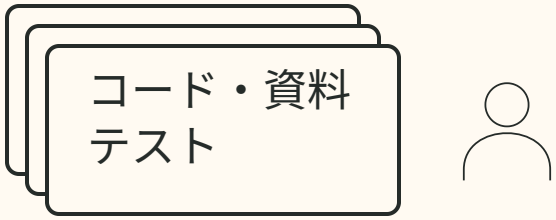
同じルールが分散

既存の処理
上長承認を維持

新しく作った処理
上長承認を維持

関係を確かめずに増える

確認する対象が増える



影響を探し、照合する
確認や修正の負担が増える

変更の影響・残作業・確認の根拠を把握しにくくなる

要件・設計・AIの作業と確認の進め方を、三つの教材で学びます

手戻りや確認負担を増やさずにAIで開発を進めるには、何が必要だったのでしょうか。

この案件で見た問題

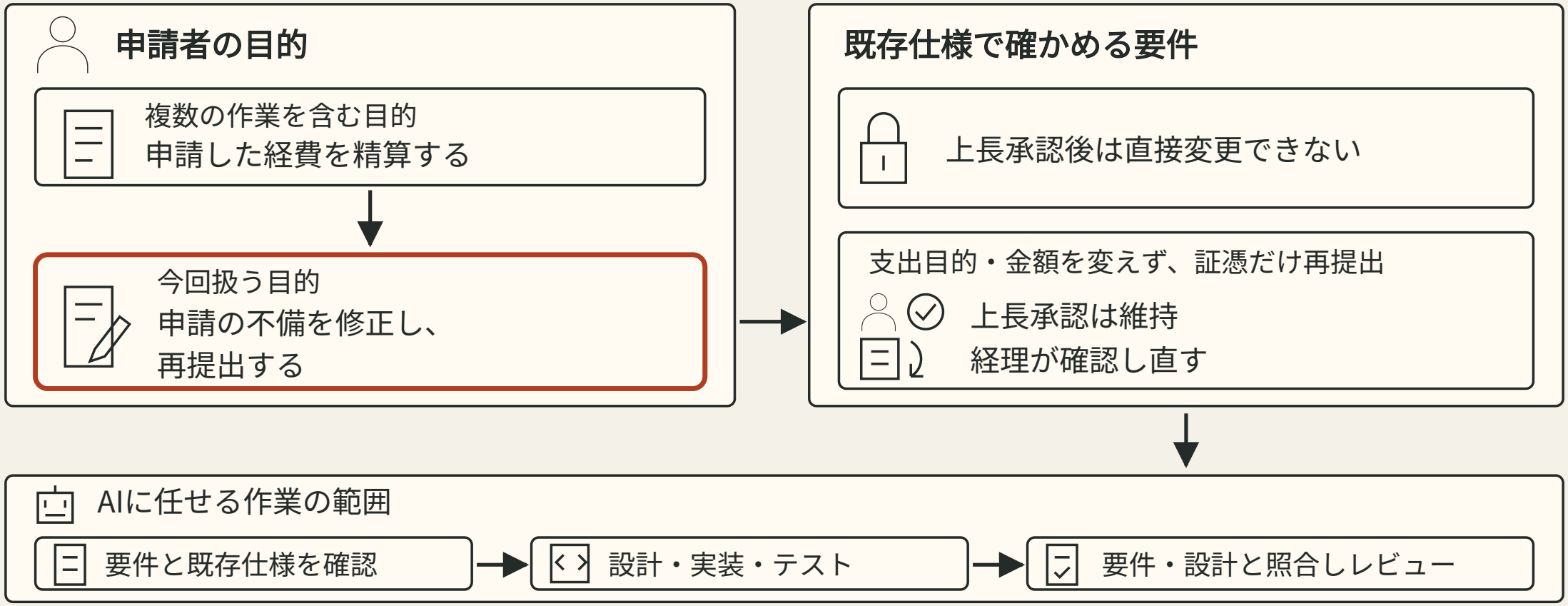
- 要件の解釈がずれた
  「承認」が指す段階を取り違えた
- 動作を直しても、
  設計の問題が残った
- 実装とレビューが
  同じ解釈で進んだ

三つの教材で学ぶこと

- 教材1
  目的と要件の明確化
- 教材2
  業務のモデル化と設計
- 教材3
  AI活用の実践と改善

教材1では、目的から必要な要件と作業の範囲を考えます

何のために、どこまでを実現するかを定め、既存の業務ルールを確かめて、AIに任せる作業へつなげます。



教材2では、業務の意味と関係をモデルに表し、設計へつなげます

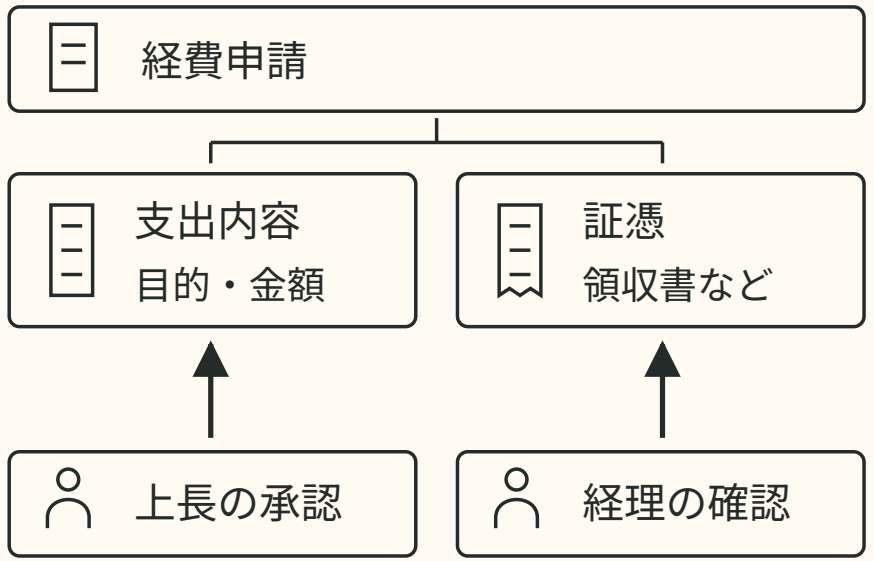
何を承認・確認したのか、何を変えると確認し直すのかを捉え、状態や処理の設計へ反映します。

教材1：目的・要件

差し戻し後の目的
申請の不備を直し、再提出する

確かめた要件
支出目的・金額不変
証憑だけを修正
上長承認は維持
経理は再確認

業務モデル：対象と判断の関係



設計に反映する例

OO　状態
支出承認と
証憑確認を区別

変更処理
証憑変更時は
経理が再確認

何を変えると、どの判断を確かめ直すかを設計に表す

教材3では、目的と設計を踏まえた作業手順をSkillsにします

Agentが各工程でSkillsを使い、要件・業務モデル・設計と照合しながら作業とレビューを進めます。

